Check for updates

DOI: 10.1111/cgf.13534

COMPUTER GRAPHICS *forum*
*Volume 38 (2019), number 1 pp. 328–339*

# TexNN: Fast Texture Encoding Using Neural Networks

S. Pratapa[1], T. Olson[2], A. Chalfin[2] and D. Manocha[3]

[1]University of North Carolina, Chapel Hill, NC, USA
psrihariv@cs.unc.edu
[2]Arm Inc.
{Tom.olson, Alex.Chalfin}@arm.com
[3]University of Maryland, College Park, MD, USA
dm@cs.unc.edu

**Abstract**

*We present a novel deep learning-based method for fast encoding of textures into current texture compression formats. Our approach uses state-of-the-art neural network methods to compute the appropriate encoding configurations for fast compression. A key bottleneck in the current encoding algorithms is the search step, and we reduce that computation to a classification problem. We use a trained neural network approximation to quickly compute the encoding configuration for a given texture. We have evaluated our approach for compressing the textures for the widely used adaptive scalable texture compression format and evaluate the performance for different block sizes corresponding to* $4 \times 4$, $6 \times 6$ *and* $8 \times 8$. *Overall, our method (TexNN) speeds up the encoding computation up to an order of magnitude compared to prior compression algorithms with very little or no loss in the visual quality.*

**Keywords:** data compression, modelling, image compression, image and video processing, GPUs and their application for general-purpose computing, hardware

**ACM CCS:** CCS Concepts •Computing methodologies→ Neural networks; Image compression; Supervised learning; Graphics file formats

## 1. Introduction

Graphics applications use texture images to store surface properties and other information used to generate visually appealing imagery. These properties can include reflectance (diffuse and specular reflectance, gloss/roughness), geometric detail (surface normals, height maps, ambient occlusion) and incident lighting (light maps) among others. Textures can consume a significant amount of GPU memory and memory bandwidth in modern graphics applications. Memory and bandwidth tend to be scarce resources, especially in mobile devices. This has led to the widespread use of lossy texture compression to reduce the consumption of limited video memory and CPU–GPU bandwidth for more than two decades [BAC96, INH99, Fen03, Ope10, SAM05].

Texture compression has become ubiquitous in graphics applications. Current compression techniques provide random access with a fixed-rate encoding rate, which maps well to the current GPU-decoding hardware. Moreover, the quality and compression rates of texture compression have improved with the introduction of newer formats, e.g. adaptive scalable texture compression (ASTC) [NLP*12]. Current formats support multiple block sizes, which increase the complexity of these formats in terms of designing efficient encoding algorithms and hardware for real-time decoding.

Many encoding methods for high-quality compression of textures are used in an offline manner. It is not uncommon for games to use hundreds or thousands of textures. Furthermore, the content development process involves multiple iterations on the texture assets. In such cases, texture compression can become a major bottleneck and can take many hours, especially on high-resolution textures [KM14b, KLM13, Don10].[1]

Current texture compression formats achieve high-quality and good compression rates by providing a broad set of flexible options and modes to compress a given texture. All these options and

[1]John Carmack: https://twitter.com/id_aa_carmack/status/503556032069517312

© 2018 The Authors
Computer Graphics Forum © 2018 The Eurographics Association and John Wiley & Sons Ltd. Published by John Wiley & Sons Ltd.

modes available for a given format are referred to as the *encoding configurations*. In many ways, the performance of a compression scheme is governed by these encoding configurations. Current encoding algorithms either use search-based methods to extensively explore the encoding configuration space or use global processing methods to speed up the search. However, computing the appropriate encoding configuration for an optimal compression scheme can be computationally intensive.

**Main results:** We present a novel approach TexNN, pronounced *tekken*, for faster encoding of textures using deep learning. The main goal is to replace the expensive search step in the texture computing pipeline and treat it as a classification problem. We train a neural network using pre-computed compressed textures and use them to efficiently compute the encoding configurations for a given texture. We design our neural network architectures to be of minimal size and train them to predict accurate encoding configurations. Furthermore, we present a scheme to integrate the trained classifier into the texture encoding pipeline (see Figure 2). Our approach can be used with any texture compression formats that use such encoding configurations. We have used TexNN to accelerate the encoding of ASTC textures and our approach can work for any block size. In particular, we highlight its results on compressed textures formats corresponding to $4 \times 4$, $6 \times 6$ and $8 \times 8$ blocks generated from Albedo textures, normal maps and bump maps. We compare the performance of our modified encoding algorithm with the original ASTC encoding scheme [NLP*12] and observe up to an order of magnitude improvement in the encoding performance. Furthermore, the compression quality is comparable to the original encoding scheme (Section 4).

We present our results as comparisons with state-of-the-art encoding algorithms and open-source implementations to encode ASTC textures [ARM12]. The original encoding scheme has multiple encoding settings (*Exhaustive*, *Fast*, etc.) for compressing a given texture. TexNN shows several benefits in different encoding settings. As compared to the *Exhaustive* setting, TexNN gives a speed-up of $7 - 10\times$ with just a small loss in quality (Figure 7). In the *Fast* setting, TexNN has similar encoding speed to original schemes with improvement in quality (Figure 10). Results for speed versus quality for different settings are shown in Figure 7 and Appendix Section 2.1 in Supporting Information. The speed-up gain from TexNN increases with increase in the resolution of the textures, as highlighted in Figure 8. The zoomed-in visual comparisons and zoomed-in visual difference comparisons in Figure 9 and Appendix Section 3 in Supporting Information show that there is little of no or little loss in perceptual quality of textures encoded using TexNN.

The rest of the paper is organized as follows: Section 2 gives an overview of background work in texture compression, texture encoding and neural networks for image compression. Section 3 gives an overview of the texture encoding pipeline and neural network topologies. We present our novel encoding algorithm in Section 4, highlighting the performance and comparing its performance with prior encoding schemes.

## 2. Background

In this section, we give a brief overview of texture compression, texture encoding methods and neural networks.

### 2.1. Texture compression

Texture compression schemes ensure the random access property by encoding non-overlapping rectangular blocks of a texture with a fixed number of bits. Delp and Mitchell [DM79] introduce a fixed bits per block encoding technique called block truncation coding (BTC) for grey-scale images. Beers *et al.* [BAC96] present the idea of compressing textures using vector quantization and list three main properties needed for texture compression scheme to be viable for real-time applications: random access, fast decompression and visual quality of decoded pixels. Modern texture compression formats follow the BTC technique. At a broad level, these formats can be classified into two categories: *table-based* schemes such as ericsson texture compression (ETC1) [SAM05] and ETC2 [SP07] and *endpoint-based* schemes, including S3 texture compression (S3TC) [INH99], powerVR texture compression (PVRTC) [Fen03], block partition texture compression (BPTC) [Ope10] and ASTC [NLP*12].

The table-based schemes store global offset value tables. For each block, a base-colour value is chosen, and for each pixel, an index into the offset table is stored. Each block of pixels is decoded by adding the offset value to the base-colour value [SAM05, SP07]. The endpoint-based schemes store two base colours per block and fix the bit weight-index values for each pixel. A block of pixels is decoded by interpolating along the line segment joining the base colours in the Red Green Blue channels of the texels (RGB) space. The base colours are also referred to as endpoint colours or endpoints.

S3TC [INH99] is the first texture compression scheme that extends the approach in Delp and Mitchell [DM79] to RGB and Red Green Blue Alpha channels of the texels (RGBA). PVRTC [Fen03] improves the quality of compression by introducing bilinear interpolation of two base colours across block boundaries. In DXT1 (S3TC), a block of $4 \times 4$ pixels is encoded into a 64-bit string that represents two 16-bit RBG565 endpoints and a 32-bit weight-index data. For a given block, DXT1 needs to compute the best RGB565 endpoints and the corresponding index data that result in the least error, i.e. optimal compression. The problem of finding the optimal endpoints that results in the least error is related to computing the shortest vector on a lattice (SVP) [KLM13]. The SVP problem is a well-known NP-hard problem [vEB81, Ajt98], and existing techniques only compute approximate solutions.

Some of the earlier texture compression schemes encoded all the pixels of a block together as a single set. Later, the idea of partitioning the pixels of a block into different sets was proposed [SP07]. Each of the colour subsets is encoded independently with some common global configuration bits across all the pixels of the block. BPTC extends the idea of partitions to the endpoint formats [Ope10]. BPTC supports a large number of partition shapes and up to three partitions for a given block. The partition shapes are pre-computed and stored in two global tables. In the ASTC format, the partition shapes are computed using a hash function, which takes a 10-bit pseudo-random number as an input [NLP*12]. Instead of using a fixed method of storing the endpoint colours like S3TC (RGB565), BPTC and ASTC also support different colour modes for storing the endpoints of each partition. Apart from colour modes, recent complex formats have several options for encoding. A given block can be compressed with a different possible combination of all

© 2018 The Authors
Computer Graphics Forum © 2018 The Eurographics Association and John Wiley & Sons Ltd.

14678659, 2019, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cgf.13534 by Nanjing University Of Science, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

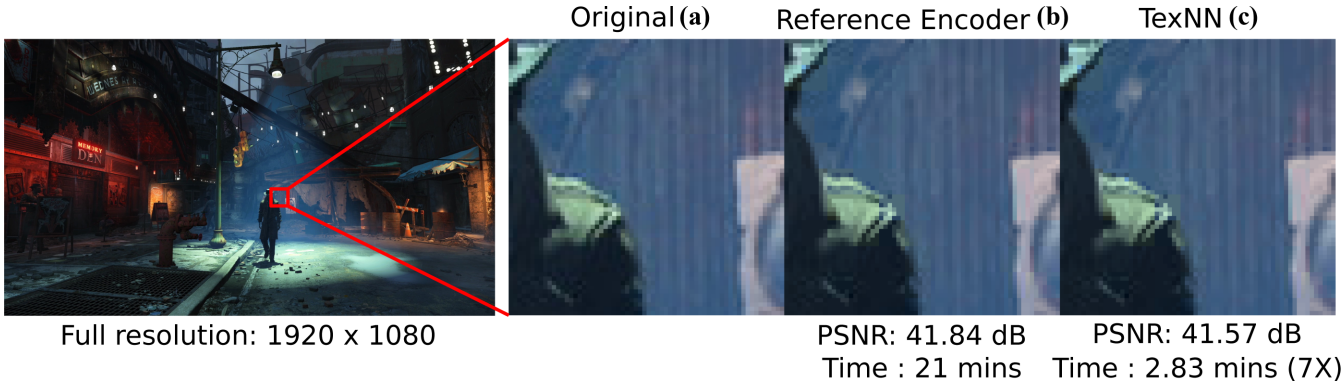



**Figure 1:** *Performance benefits of our TexNN encoding algorithm: (a) Original texture. (b) Texture encoded using original ASTC encoder in exhaustive setting. (c) Texture encoded using TexNN-based encoder. We observe* $7-10\times$ *speed-up in encoding time with very little loss in image fidelity.*

additional encoding options present. We refer to the combinations of all possible encoding options as *encoding configurations*. One of the primary aims of an encoding algorithm is to determine the right encoding configuration for the given input and our approach reduces this to a classification problem.

### 2.2. Texture encoding

All the texture data required in an application are typically compressed using an offline algorithm and stored for later use while rendering. The primary aim of the encoders is to produce optimal encoding, for a given block. Every encoding uses trade-off settings between encoding speed and compression quality; the faster the encoder, the worse the quality. Several offline encoders are available for compressing into S3TC, PVRTC and BPTC [Don10, Blo09, Bro06]. Few real-time encoders exist for encoding into S3TC format [Wav06, WCn07]. Recently, a few methods targeted at PVRTC and BPTC have been introduced, and they speed up the encoding by processing the whole texture to extract useful global information for compressing the blocks independently [KM14a, KM14b]

A method for compressing into PVRTC was developed in which the entire texture is pre-processed with intensity dilation to generate high- and low-frequency components that are used in computing the endpoints [KM14a]. In SegTC [KM14b], the texture is prep-processed using a global segmentation method to determine the best partitions for each block. Good real-time encoders exist for compressing into S3TC, PVRTC and ETC formats, but only a few exist for complex formats such as BPTC. No real-time encoders are known for ASTC because of its complexity. The primary encoder used for compressing ASTC is the ARM encoder [ARM12]. In the ARM encoder, the exhaustive compression mode explores all possible encoding configurations using branch and bound heuristics to pick an encoding configuration for optimal compression. NVIDIA ASTC compression tools, based on CUDA, utilize GPUs parallelism [EN15]. The NVIDIAs tool uses a brute force approach in which all the blocks are encoded using all possible encoding configurations in parallel. For each block, the configuration that results in optimal compression is chosen. Our goal is to replace these exhaustive searches by a classification problem.

### 2.3. Neural networks for image compression

Neural networks have been proved to be incredibly useful for some challenging tasks such as object recognition, speech translation, machine translation and video captioning.

In these applications, neural networks have been shown to be good at extracting useful information and features from the data but can also make the right decisions based on the obtained information. The initial success of the neural networks motivated their application to a wider range of problems, including image compression. The main idea is to design neural networks that can imitate standard image compression techniques such as JPEG [Wal92], JPEG2000 [SCE01] with learning methods to get better compression ratios or quality. Toderici *et al.* [TVJ*16], Tao *et al.* [TJZ*17] and Rippel *et al.* [RB17] describe an end-to-end image compression–decompression system based on neural networks. These approaches use unsupervised learning method since they need both compression and decompression capabilities. While our approach is motivated by the recent work on image compression, the underlying problem of texture compression is quite different from that of image compression. Since the texture decoding hardware needs random-access to the texture data, current texture compression schemes use fixed compression rates. This is in contrast to image compression formats like JPEG or PNG, which use variable-length encoding techniques, resulting in much higher compression rates as compared to texture compression schemes. As a result, the issues in the design of neural networks for texture compression are quite different from their use for image compression and we present a new supervised learning approach to accelerate the encoding.

## 3. TexNN: Our Approach

In this section, we give an overview of texture encoding schemes, address many issues in the design of neural networks and describe our approach to training the network.

© 2018 The Authors
Computer Graphics Forum © 2018 The Eurographics Association and John Wiley & Sons Ltd.

14678659, 2019, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cgf.13534 by Nanjing University Of Science, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

### 3.1. Texture compression using neural networks

Recent texture compression formats that generate high-quality textures need to deal with three main challenges as part of the encoding scheme. These include a large set of block partitions, multiple colour encoding modes for storing base colours and different bit precision and scaling for storing the weight-index values. The possible solutions to each of these components increase the total possible set of encoding configurations by a large factor. This significantly increases the complexity of a format and encoding into the format [EN15].

The random access property is ensured by the texture formats by having a fixed bit rate encoding per block. Due to the fixed bit budget per block, the encoding configurations are constrained to a set of possible discrete values. Each possible encoding configuration determines how the available bits are distributed for storing partitions, base colours and weight-index data. For example, to encode a block with a single partition, there is no need to store any partition information that provides more bits for base colours or weight-index values. In the case of multiple partitions, the base colours of each partition can be stored in different colour modes. Different colour encoding modes require a different number of bits to store the colours. This helps in storing colours with fewer details in a small number of bits, and thereby offers more bits for weight-index values. The trade-off between the bits allocated between different encoding configurations determines the compression quality of a block.

For a given set of $n$ possible encoding configurations $\mathbf{EC}$ and an input block of pixels $\mathbf{B}_{in}$,

$$\mathbf{EC} = \{ec_1, ec_2, \ldots ec_n\}, \tag{1}$$

$$\mathbf{B}_{ec_i} = encoder(\mathbf{B}_{in}, ec_i), \tag{2}$$

$$Error_{ec_i}(\mathbf{B_{in}}) = \|\mathbf{B}_{in} - \mathbf{B}_{ec_i}\|. \tag{3}$$

$\mathbf{BC}_{ec_i}$ is the compressed block of pixels encoded using $ec_i$ and $Error_{ec_i}$ is the corresponding error. The goal of an encoding algorithm is to determine the optimal encoding configuration from the $EC$ set:

$$ec_{opt}(\mathbf{B_{in}}) = \min_{\forall ec \in \mathbf{EC}} (Error_{ec}(\mathbf{B_{in}})). \tag{4}$$

Existing algorithms tackle the computation of $ec_{opt}$ by searching over the $EC$ set. As shown in Figure 2, we restate the problem of finding optimal encoding configurations as a classification problem because there are only fixed a number of possibilities for each encoding option.

$$\mathbf{C} = classifier(\mathbf{B}_{in}; \mathbf{EC})\ \mathbf{C} \subset \mathbf{EC}. \tag{5}$$

We use a classifier to predict a small set of good encoding configurations $\mathbf{C} \subset \mathbf{EC}$ and use them to compress the input block $\mathbf{B}_{in}$. Neural networks have been established as one of the best methods for a broad range of classification problems in different fields [WBX*07, BSP92]. In the rest of the paper, we present our approach using ASTC as our target texture compression format. In many ways,

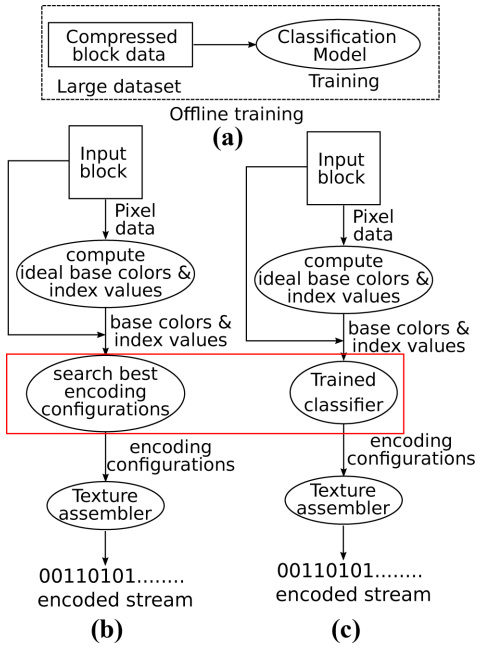



**Figure 2:** *Fast texture encoding using TexNN: (a) We highlight the offline learning pipeline that computes the classification model. (b) Overall pipeline of current texture encoding methods (e.g. ASTC), which performs an expensive search step over encoding configurations. (c) We highlight the TexNN-based encoding pipeline, which uses the trained neural network to infer encoding configurations to speed up the encoding process.*

ASTC is regarded as one of the most complex formats, and any technique used to accelerate the encoding of ASTC should also be applicable to other formats. In particular, we train neural networks for computing the encoding configurations for ASTC. The training of the neural networks on large data takes time and is performed offline. The training step needs to be performed only once and the trained networks can be used for inferring good encoding configurations for fast encoding.

### 3.2. Adaptive scalable texture compression

ASTC supports block sizes starting from $4 \times 4$ size to $12 \times 12$ size, with bit rates varying from 8 bpp down to 0.89 bpp, respectively [NLP*12]. ASTC uses many techniques such as partition hash function, bounded integer sequence encoding (BISE) and scaling of weight-index values to provide better compression quality at low bit rates. ASTC supports a large number of fixed partition shapes for a given block size. The partition shape is determined by the 10-bit partition index and 2-bit partition count. It has a dual plane mode that allows independent encoding of some channels of a given block. Each of the base colours for each partition can be stored in different colour modes. Instead of storing all the per-texel weight-index values, the 2D grid weight-index values are scaled down to a smaller size. While decoding, the scaled 2D grid is bi-linearly interpolated to regenerate the original weight-index grid. The amount of scaling performed per each block is referred to as the weight decimation

© 2018 The Authors
Computer Graphics Forum © 2018 The Eurographics Association and John Wiley & Sons Ltd.

14678659, 2019, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cgf.13534 by Nanjing University Of Science, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

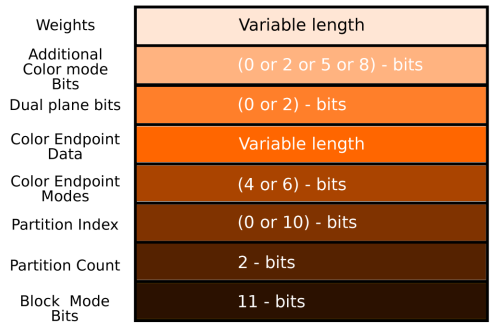



**Figure 3:** *A representative structure of a 128 bit compressed ASTC block. Different fields and the corresponding bits allocated are marked. For efficient decoding, the encoding configuration bits are interleaved with base-colour and weight-index bits.*

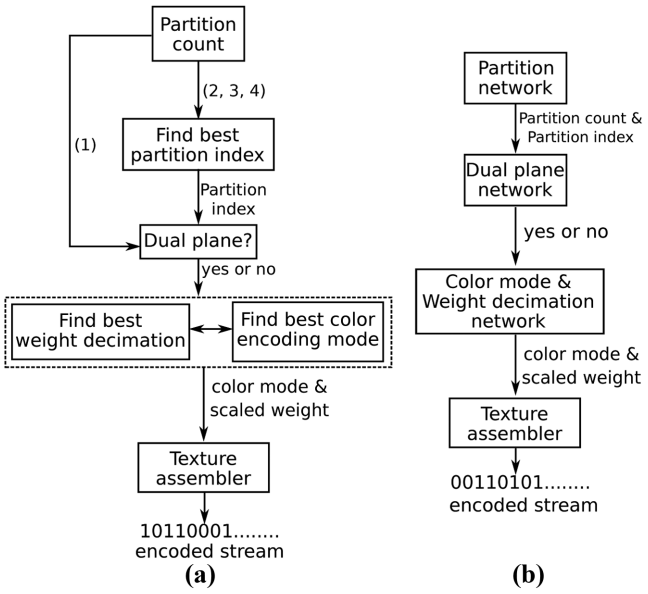



**Figure 4:** *Comparison between the reference ASTC encoder pipeline [ARM12] with our TexNN encoding pipeline similar to Figure 2. (a) The encoding flow and several stages of the reference encoder used to compute the encoding configuration. (b) Our TexNN encoding pipeline for ASTC using the trained neural networks. We replace the original functions to compute the encoding configurations with trained neural networks. Reference encoder spends 80% of the time is spent in searching for good encoding configurations. Using our TexNN pipeline, we accelerate the encoding process by using neural networks.*

mode. The quantized base-colour and weight-index values are further contracted using BISE. Figure 3 shows the possible structure of a 128-bit compressed ASTC block with different fields marked for the base colours, weight index and encoding configurations mentioned above.

Figure 4(a) shows the pipeline for the reference encoder [ARM12]. The figure shows the flow of the encoder in terms of

**Table 1:** *Accuracy-10 of our trained neural networks P-NN & DEM-NN on the validation data set. The resulting accuracy of the network depends on several training parameters. The networks are trained using the Tensorflow framework on NVIDIA-TITAN PASCAL GPU.*

| | Accuracy-10 | |
|---|---|---|
| Block size | P-NN | DEM-NN |
| $4 \times 4$ | 65 % | 95 % |
| $6 \times 6$ | 58 % | 92 % |
| $8 \times 8$ | 52 % | 84 % |

**Table 2:** *Breakdown of time taken in different steps shown in Figure 4 for encoding into ASTC using ref-encoder. The timing values are for a 2K texture in exhaustive mode for a block size of $4 \times 4$. Major part of the encoding time (80%) is spent in searching for partition, endpoint mode and weight decimation mode. Our method (TexNN) replaces the expensive time taking search step with inference from neural networks for faster encoding. The encoding timings are measured on an Intel(R) Xeon(R) CPU, 2.4 GHz using a single thread.*

| Step | Ref-encoder (s) |
|---|---|
| Partition search | 485.62 (50%) |
| Ep & Wt. mode search | 330.22 (34%) |
| Ep & Wt. value computation | 154.77 (16%) |
| Total time | 970.61 |

computing the encoding configurations. The reference encoder has different functions to determine the best possible encoding configurations, base colours and weight-index data. The reference encoder performs an extensive search using branch and bound on heuristic error measures. We train neural networks to mimic the main functions of the reference encoder: *find best partition function*, *decide dual plane*, *find best weight scaling* and *find best colour encoding mode*. Once the proper encoding configurations are determined, generating the compressed bit string is straightforward. Brute force encoding is possible for determining the best possible encoding configurations since the configuration space is discrete. Exploring the entire configuration space can be computationally expensive [EN15]. Table 2 has approximate breakdown of time spent by the state-of-the-art encoder in several important steps of the encoding process.

### 3.3. Neural network design

In this section, we give an overview of the neural network that is used for the classification step at runtime. This problem can be formally stated using the following formulation:

$$\mathbf{Pred} = NNTrain(\{\mathbf{B}_{in}\}, \{ec_{opt_i}\}, \Theta), \qquad (6)$$
$$\mathbf{Error} = Loss(\mathbf{Pred}, \{ec_{opt_i}\}).$$

The *NNTrain* function takes several inputs, as shown in Equation (6), where $\{\mathbf{B}_{in}\}$ :input training blocks, $\{ec_{opt_i}\}$ :ground truth

© 2018 The Authors
Computer Graphics Forum © 2018 The Eurographics Association and John Wiley & Sons Ltd.

14678659, 2019, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cgf.13534 by Nanjing University Of Science, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

encoding configurations, $|\Theta|$ :size of the neural network and $Loss$ :minimization function used to tune $\Theta$. The output of the trained network is a probability distribution **Pred** over the possible encoding configurations **EC** where,

$$\textbf{Pred} : \{\text{Prob}(ec_1), \text{Prob}(ec_2), \ldots \text{Prob}(ec_n)\},$$

with **pred** is the probability distribution over **EC** and Pro$b(ec_i)$ :Probability that $ec_i$ is a good encoding configuration.

The parameter values $\Theta$ for the neural network are randomly initialized using a standard probability distribution (e.g. Gaussian, Bernoulli, etc.). We use Gaussian distribution to initialize our networks for training. The $Loss$ function takes the predicted probability distributions **Pred** and ground truth labels as input and outputs an error measure related to their closeness. *NNTrain* utilizes the error from $Loss$ function for tweaking parameter values $\Theta$ over all the training data until the error converges to a minimum (Equation (7)). The network is trained for a certain number of steps and the convergence to minimum is manually checked by evaluating the slope of the accuracy curve with time.

$$\min_{\hat{\Theta}}(Loss(\textbf{Pred}, \{ec_{opt_i}\})). \tag{7}$$

For a given input block $\mathbf{B}_{in}$, let $top_k$ set be the most probable $k$ predictions ($k = [1, 2, 3, \ldots]$) in **pred** from the network. If the actual ground truth label is found in within $top_k$ set, we consider it as a $top_k$ *hit*. Prediction $accuracy_k$ for a trained neural network over a set of validation data set is defined as the ratio of number of $top_k$ hits to total number of predictions computed (Equation (8)).

$$accuracy_k = \frac{\#(top_k \text{ hits})}{\#(\text{total predictions})}, \tag{8}$$

$$\textbf{Pred} = NNClassify(\{\mathbf{B}_{in}\}, \hat{\Theta}). \tag{9}$$

Once *NNTrain* converges to a minimum, we can use the converged final parameters $\hat{\Theta}$ to compute the encoding configurations for new input blocks. *NNClassify* function (Equation (9)) takes in new input blocks $\mathbf{B}_{in}$ and the trained $\hat{\Theta}$ from *NNTrain* and outputs the **Pred**, probability distribution over possible encoding configurations **EC**.

The primary encoding configurations (**EC**) needed to be determined in case of ASTC are the following: (*partition count*, *parition index*, *endpoint colour mode* and *weight decimation mode*). We use approach described to design and train appropriate *NNTrain* function with required training data inputs, the right $Loss$ function and proper network parameters $\Theta$ for determining the above encoding configurations.

### 3.4. Practical issues in network design

The choice of loss function (Equaution (7)) is critical in determining if a network is being trained for the right task. Loss function also affects the final outcome of the network. The ideal error function for the current task would be the mean square error (MSE) between the ASTC compressed blocks with ground truth configuration and the configuration predicted by the network. For a given input training block $\mathbf{B}_{in}$, let $\mathbf{MSE}(ec_i)$ represent the resulting mean square error if the input block is encoded using $ec_i$. To use **MSE** as the loss measure, we need to compute the error as $Error = \|\mathbf{MSE}(ec_{pred}) - \mathbf{MSE}(ec_{opt})\|$ where $ec_{pred}$ is the top prediction in **Pred** by the network. The computed error *Error* is used to tune the network parameters $\Theta$.

$$\textbf{Pred} : \{\text{Prob}(ec_1), \text{Prob}(ec_2), \ldots \text{Prob}(ec_n)\}.$$

Since we do not know the **Pred** for a input block $\mathbf{B_{in}}$, we need to compute

$$\mathbf{MSE}(EC) = \{\mathbf{MSE}(ec_1), \mathbf{MSE}(ec_2), \ldots \mathbf{MSE}(ec_n)\}$$

for all possible encoding configurations **EC** and include that as part of the labelled training data for all the blocks. The total size of the set **EC** for ASTC is ∼1–2 million.[2] We believe that this task of generating the final compressed block for all possible configurations would be intractably long and practically infeasible given the number of total encoding configurations possible. We treat the problem of picking good configuration as a classification problem over possible configurations and use cross-entropy to distinguish different configurations. We base this on the observation that the ground truth encoder picked a set of optimal configurations that resulted in the low MSE value for a given input block. Therefore, deciding on good encoding configurations similar to the ground truth encoding configurations will effectively result in final lower MSE and our results demonstrate the same.

The trained network should predict a good encoding configuration from **EC** that gives all the parameters (partition size and index, dual-plane mode, weight scaling and colour mode) required for encoding discussed in Section 3.2. As mentioned earlier, the set **EC** for ASTC is ∼1–2 million in size. The parameter size of the network required for such big classification task will be very large. Data required to train such a big network might be 10× the size of our current training data set and could result in over-fitting problems. Hence, we separately train different neural networks for inferring different encoding parameters.

### 3.5. Network topologies

Given the problem specification in the previous section, our goal is to design the appropriate network topology. An ideal design would correspond to a minimal size network, i.e. small $|\Theta|$, that results in good final $accuracy - k$. We have designed different neural networks to compute several encoding configurations, including fully connected neural networks, convolutional neural networks and

[2]Total possible pairs of partition count and index: 3069. Total possible pairs of weight-index and base-colour modes depend on the block size and vary between 1300 and 7300. Computing the cross product of the two sets results in about 3M–21M possible configurations. By removing all the invalid configurations that are not supported in ASTC results in a conservative estimate of 1–2 million valid encoding configurations.

© 2018 The Authors
Computer Graphics Forum © 2018 The Eurographics Association and John Wiley & Sons Ltd.

14678659, 2019, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cgf.13534 by Nanjing University Of Science, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

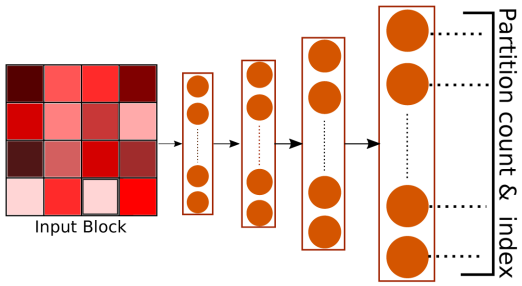



**Figure 5:** *Partition neural network (P-NN): Fully connected network structure for computing the partition count and partition label. The input to this network is the input block of the texture that needs to be compressed. The output of the network is a probability distribution over all possible partition labels. We select the $top_k$ probable labels from the output and use them for encoding.*

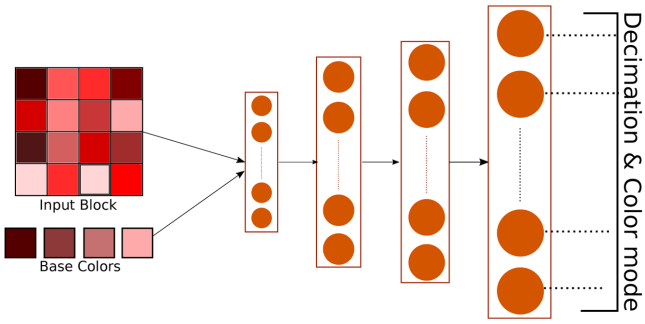



**Figure 6:** *Decimation and endpoint mode neural network (DEM-NN): Network structure for computing the decimation and colour mode for storing the quantized weight-index and base-colour values. The input to this network is the block of a texture and corresponding base colours computed for that block. It computes a probability distribution over a set of decimation and colour modes allowed in the encoding format. $Top_k$ of most probable configurations are selected for encoding.*

a dual loss function to be used for encoding as shown in Figure 4(b). More information about network sizes and network structures is given in Section 1 of the Appendix in Supporting Information. In our experiments, we observe that fully connected neural networks work well in terms of computing the encoding configurations. In the fully connected networks, all neurons in a given layer are connected to all the neurons in the next layer. This is due to the fact that the input block sizes are quite small and the number of pixels is about a few hundred. The convolutional networks are particularly useful when there is large amount of image data and high-level information needs to be extracted for further processing [SRASC14].

**Partition index and count:** Figure 5 shows the structure of the network trained for inferring partition count and the partition index for ASTC. The input to the network is the pixels of the block. Each single output label of this neural network represents a pair of possible partition count and partition index. The labels to outputs of the network are assigned in increasing lexicographical order of partition count and partition index pairs. The final output is the probability distribution over all the output labels of the network. We refer to this trained neural network as *partition neural network (P-NN)*.

**Decimation and endpoint mode:** Figure 6 shows a network structure used for inferring the endpoint mode and weight decimation. Once the ideal base colours and weight-index values are computed for a given block, they have to be quantized and stored in a fixed number of bits. The colour endpoint mode determines the number of bits used for base-colour values and the weight decimation mode determines the bits to be used for weight-index values. Computing the right trade-off of bit allocation between base-colour and weight-index values has an impact on the resulting quality. This implies that the inferences of decimation mode and endpoint mode cannot be made independently. Taking this into account, we train a network to determine both the encoding options in tandem. In addition to the block colours, we input the computed base colours for all the partitions to the colour mode and weight decimation network. Each output label of this network, as shown in Figure 6, corresponds to a possible pair of base-colour modes and weight-index decimation modes. The labels for the outputs of the network are assigned in an lexicographic order of pairs of base-colour modes and weight-index decimation modes. The final output of the network is a probability distribution over all the output labels of the network. The assignment of the additional input of computed base-colour values improves the network's prediction accuracy. We refer to this trained neural network as *decimation and endpoint mode neural network (DEM-NN).*

### 3.6. Training and evaluation of networks

The performance and the resulting accuracy of a network depends on the loss functions and several initial training parameters known as the hyper-parameters. The primary hyper-parameters that determine the performance of a network are the batch size, learning rate and activation function. We ran a total of 50 experiments with varying network sizes and hyper-parameters. We observe that a batch size in the range of 100–200 and learning rates of the order of $10^{-4}$ results in stable training of the network and good classification accuracy. We use two types of activation functions relu [NH10] and selu [KUMH17]. Selu is a recently introduced activation technique useful for training fully connected networks with a self-normalizing technique that results in good prediction accuracy for the network [KUMH17]. We use cross entropy [SJ81] as the loss function for our training. We minimize the cross-entropy loss by changing the network weights over each batch size by using a gradient descent minimization approach [KB14]. To avoid the major problem of over-fitting caused in neural networks, we used the technique of dropout [SHK*14] with a dropout value of 0.6.

We gathered over 2050 LDR RGB textures from different sources [Fra99, RDS*15] for training the neural networks. In our training set, we have a mixture of albedo textures, normal maps and bump maps. Each block of the texture is encoded independently, so that each block results in a ground truth label that can be used for training. We used the state-of-the-art encoder [ARM12] to

© 2018 The Authors
Computer Graphics Forum © 2018 The Eurographics Association and John Wiley & Sons Ltd.

14678659, 2019, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cgf.13534 by Nanjing University Of Science, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

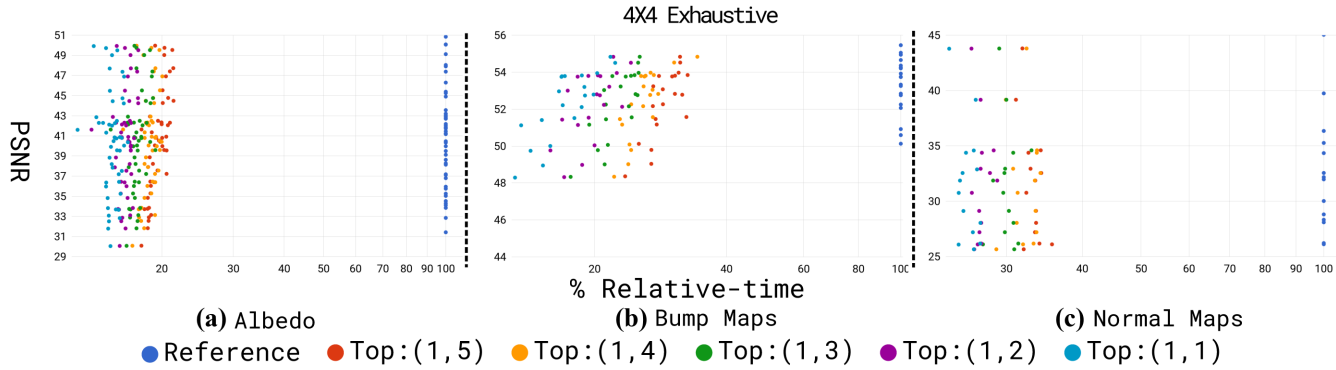



**Figure 7:** *The encoding time versus quality comparison between TexNN and ref-encoder on different texture modalities in exhaustive setting for block size 4 × 4. The horizontal axis represents the relative time in percentage, showing percentage of time taken using TexNN in several modes w.r.t. reference encoder. Each data point corresponds to an image in the test data set compressed using a particular encoding setting. All the points in one colour correspond to textures encoded using a particular setting. We use relative encoding time to highlight the improvement. For each comparison, we fix the setting to one of the ref-encoder settings and vary TexNN settings from top:(1, 1) to top:(1, 5). In all the comparisons, we observe small variation in PNSR. We also highlight the results for different category of textures separately (albedo textures (a), bump maps (b) and normal maps (c)). We observe similar results for all three texture categories used in our experiments. The encoding speeds are measured on an Intel(R) Xeon(R) CPU, 2.4 GHz using a single thread.*

**Table 3:** *(a) Performance of inferencing an encoding configuration from the trained P-NN & DEM-NN. We present the average times for different block size. The total time taken increases with the size of the network (Θ) due to the increased number of computations. These inference times are measured on an Intel(R) Xeon(R) CPU, 2.4 GHz. (b) Average speed gained over all the blocks test image data set using exaustive-top:(1, 1) setting using both the networks. The speed-up gained goes up with bigger block sizes as the complexity of reference encoder increases with the number of input pixels.*

| | (a) Inference time (ms) | | |
|---|---|---|---|
| Block size | P-NN | DEM-NN | (b) Speed-up |
| 4 × 4 | 0.079 | 0.085 | 6.38 |
| 6 × 6 | 0.20 | 0.28 | 6.57 |
| 8 × 8 | 0.38 | 0.654 | 7.66 |

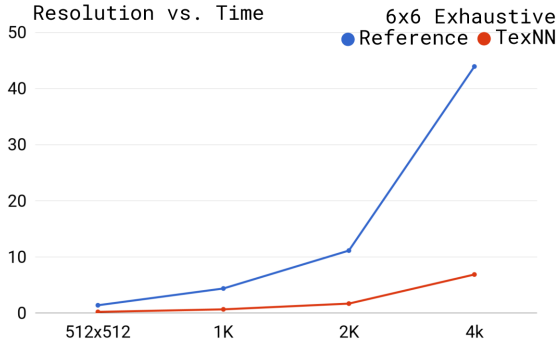



**Figure 8:** *Comparison of increase in average encoding time for several resolutions between ref-encoder and TexNN. The textures are encoded in exhaustive setting of the ref-encoder and the setting for TexNN is top: (1, 1). We observe that higher the resolution, more the speed-up gained by TexNN.*

generate the ground truth labels. We trained neural networks for block sizes 4 × 4, 6 × 6 and 8 × 8. The total number of labelled ground truth blocks data from 2050 images are 4.5 million 4 × 4 blocks, 3.3 million 6 × 6 blocks and 2.1 million 8 × 8 block labels. We have approximately 950K (4 × 4), 700K (6 × 6) and 440K (8 × 8) blocks of normal maps and bump maps blocks, respectively, in the training set. The labelled data are randomly separated into training (85%) and validation (15%) sets. We use the training data set for batch training of the neural networks. Validation data are used to verify the convergence of neural networks towards a minimum during the training phase. The exact training time varies from network to network depending on the size of our model. The average training time for each of our networks is approximately 40–48 h on one NVIDIA Pascal GPU. Table 1 shows the $top_{10}$ accuracy of the trained networks we have used in our TexNN encoder for final evaluation.

## 4. Evaluation and Analysis

In this section, we describe the results from extensive evaluation of our approach and analyse its performance. In order to test TexNN, we collected 89 textures consisting of 48 Albedo textures, 21 bump maps and 20 normal maps of various resolutions, varying from 512 × 512 to 3840 × 2160. The test images are collected separately from the Pixar texture data set [Pix15], RetargetMe data set [RGSS10] and publicly available high-resolution screen shots from a game [Stu17]. The textures used in the final test data set to evaluate the performance of TexNN-based encoder are different and not included in the training or validation sets with respect to the classification model. We compare the quality and encoding speed using TexNN-based encoding of ASTC with the existing ASTC reference encoder (ref-encoder) from ARM [ARM12]. The ref-encoder has

© 2018 The Authors
Computer Graphics Forum © 2018 The Eurographics Association and John Wiley & Sons Ltd.

14678659, 2019, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cgf.13534 by Nanjing University Of Science, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

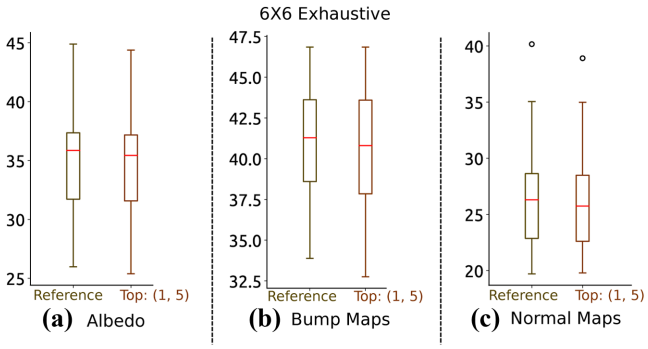



**Figure 9:** *Comparison of the PSNR values for the test set encoded using reference encoder and TexNN using a box and whiskers plot. The comparison for the three modalities, albedo (a), bump maps (b) and normal maps (c), are presented separately. The quantitative comparison shows that the compression quality (PSNR) is similar for TexNN and reference encoder.*

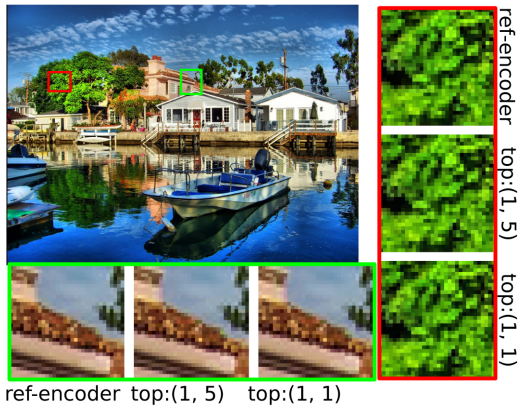



**Figure 10:** *An interesting* $32 \times 32$ *regions of a test image are cropped and scaled to* $512 \times 512$ *to compare visual quality of compressed textures generated by ref-encoder with TexNN in some settings.*

five quality versus speed trade-off encoding settings, *exhaustive*, *thorough*, *medium*, *fast* and *very fast*. *Exhaustive* setting of the encoder gives the best quality and takes the most time. On the other hand, *fast* setting is $10 - 50X$ faster compared to exhaustive setting but the compression quality reduces by 2–3 dB, which can be visibly significant.

In the TexNN encoder, for each block, we infer the $top_k$ most probable encoding configurations from the neural network and encode the block using the inferred configurations. The block is compressed in all inferred $top_k$ configurations, the configuration that results in the least MSE for the block. We preserve the encoding settings of the ref-encoder in our TexNN encoder. We compare the encoding results from TexNN with those of ref-encoder with different settings. These settings correspond to:

- *ref-setting* encoding performed using ref-encoder [ARM12] in a given *setting*.
- *setting-top:(n1, n2)* refers to the encoding performed using TexNN with $top_{n1}$ and $top_{n2}$ inferences from P-NN and DEM-NN networks, respectively, in a given *setting*.

Since there is a large number of combinations of the encoding settings, we present some of the results in this section. Other detailed results are provided in Section 2 of the Appendix in Supporting Information. The inference step from the networks in TexNN can be parallelized on GPUs and accelerate our method further. The ref-encoder is currently available for running on CPUs only. In order to perform fair comparisons between the encoding times of TexNN and ref-encoder, we presented results done on a CPU with single thread.

For a given encoding setting, we define relative time $= \frac{time(encoder)}{time(ref-encoder)}$. We use relative time to normalize the encoding time across textures of different resolutions in the test data set. Scatter plots are used to present a clear comparison between TexNN and

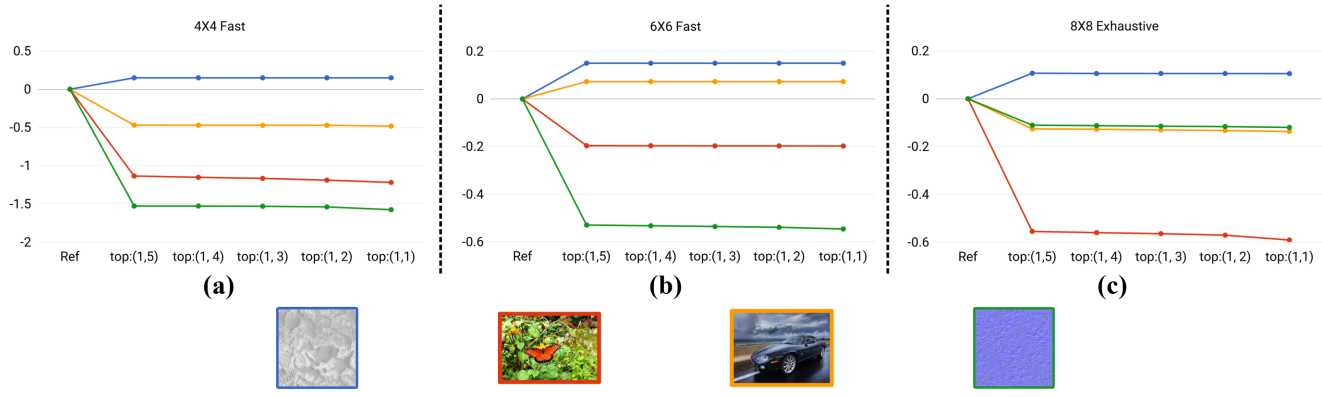



**Figure 11:** *We plot the change in PSNR for different settings of TexNN from ref-encoder for a few textures from the test data set. We show the changes for exhaustive and fast settings for* $4 \times 4$, $6 \times 6$ *and* $8 \times 8$. *Part (a) shows the plot for* $4 \times 4$ *block size. Part (b) plots the change for* $6 \times 6$, *and we notice a little increase in quality using TexNN in the fast setting. We also notice a little increase in quality for* $8 \times 8$ *in the fast setting, as seen in part (c). We observe this small increase in quality across few images for different block sizes in some encoding settings.*

© 2018 The Authors
Computer Graphics Forum © 2018 The Eurographics Association and John Wiley & Sons Ltd.

14678659, 2019, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cgf.13534 by Nanjing University Of Science, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

ref-encoder for textures in the test data set. All the textures in the test data set are compressed using the ref-encoder and TexNN in different encoding settings. The encoding time and the resulting quality is noted for all images in each of the encoding settings. Figure 7 shows the scatter plot comparison of TexNN with ref-encoder in terms of relative time and the resulting compression quality. The horizontal axis in Figure 7 represents relative time in terms of a percentage measure w.r.t. to total encoding time taken by the ref-encoder. As a result, in the plot (Figure 7), textures encoded using ref-encoder are shown as 100% and textures encoded using TexNN take small fraction of time with respect to the ref-encoder. All the points in the plot with the same colour correspond to textures compressed in a particular setting.

We use peak-signal-to-noise ratio (PSNR) as our quality metric to compare the compression results. We observe speed-ups of $8 - 10\times$ in the *exhaustive* and *thorough* settings [ARM12] for all the block sizes, with a PSNR loss of 0.3–0.5 dB. For *medium*, *fast* and *very fast* settings [ARM12], we observe that the ref-encoder performs as well or better than TexNN. We observe similar results (quality and speed) in all of the three modalities we have used. The performance measurements for TexNN and ref-encoder are generated on a CPU running with a single thread.

We highlight the change in the compression quality between ref-encoder and TexNN for different encoding settings in Figure 11. Moreover, we notice that in some cases of the encoding settings, the compression quality of TexNN is better. This suggests that the neural network that results in better accuracy can improve the compression quality of TexNN-based encoder. The speed-up obtained using TexNN increases as the block size increases. The time complexity of the search algorithms for finding encoding configurations increases with a number of input pixels. We highlight these speed-ups in Table 3(b) by computing the average speed-up per block across all the blocks in the test data set. The speed-up gained from TexNN improves with higher resolution textures, we point to this observation in Figure 8. A quantitative quality (PSNR) comparison for the test set using a box and whiskers plot for TexNN and ref-encoder for different modalities is highlighted in Figure 9. Similar box and whisker plots for different block sizes and encoding settings are included in the Section 2.3 of the Appendix in Supporting Information. In Figure 10, we show scaled-up comparisons of compressed textures. The comparison shows that the visual quality does not differ between compressed textures encoded using ref-encoder and TexNN. We present more detailed visual quality and visual difference comparisons in Section 3 of the Appendix in Supporting Information.

## 5. Conclusion, Limitations and Future work

**Conclusion:** We present a new method (TexNN) for fast encoding of textures using deep learning. Our approach trains a neural network using ground truth data and uses that as a classification model at runtime. We replace the expensive search step in current encoders and observe up to an order of magnitude performance improve for ASTC format, with very little or no loss in visual fidelity.

**Limitations:** One of the limitations of our approach is that we rely on large amounts of data required to train the neural networks. The classification accuracy of the neural networks is dependent on the variation present in the training data. Adding different kind of textures such as normal maps, bump maps and light maps can increase the variation in the training data. The generation of ground truth labels for training data is another limitation in our approach because the accuracy depends on currently available encoders. Furthermore, our approach is mainly suited for compression methods where the search step is a major bottleneck, and may not help encoding algorithms for other formats, e.g. DXT.

**Future work:** Although our demonstration is currently limited to ASTC, it can be applied to BC6H or BC7 in a straightforward fashion. Our neural networks can be quickly retrained to infer encoding configuration for other complex formats (BC6H or BC7). There are several ways to further improve both the compression quality and speed of encoding of our method. Inference computations on neural networks are well suited for implementation on GPUs [OJ04]. The entire encoding process can be parallelized on a GPU as all the blocks for a texture are encoded independently, and it may be possible to obtain another order of magnitude improvement in the running time. This has the potential of being the first real-time algorithm of high-quality texture compression on mobile and desktop GPUs. The compression quality can be further improved by designing and training neural networks that predict the right encoding configurations more accurately. Different modalities exhibit different properties and training different networks separately for each modality might improve the compression quality and needs further investigation. As pointed out in Section 3.5 if we can generate labelled training data with MSE values, the accuracy might be better. Actual ground truth encodings for the training data can be computed by absolute brute force approach independent of the current encoders. Independent ground truth labels help the neural networks predict the one close to actual ground truth improving the quality. End-to-end compression and decompression system using auto-encoders [RB17] is a good topic for future work.

## Acknowledgements

This research is supported in by ARO grant W911NF-18-1-0313, Google and Intel.

## References

[Ajt98] AJTAI M.: The shortest vector problem in l2 is np-hard for randomized reductions (extended abstract). In *STOC '98: Proceedings of the 30th Annual ACM Symposium on Theory of Computing* (New York, NY, USA, 1998), ACM, pp. 10–19.

[ARM12] ARM: Astc evaluation codec. https://github.com/ARM-software/astc-encoder, 2012. Accessed on 20 June 2017.

[BAC96] BEERS A. C., AGRAWALA M., CHADDHA N.: Rendering from compressed textures. In *SIGGRAPH '96: Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques* (1996), ACM, pp. 373–378.

[Blo09] BLOOM C.: Oodle, distributed with the granny3d package from rad game tools. http://www.radgametools.com/granny.html, 2009. Accessed on 1 July 2017.

© 2018 The Authors
Computer Graphics Forum © 2018 The Eurographics Association and John Wiley & Sons Ltd.

14678659, 2019, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cgf.13534 by Nanjing University Of Science, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

[Bro06] Brown S.: libsquish. https://github.com/svn2github/libsquish/, 2006.

[BSP92] Bischof H., Schneider W., Pinz A. J.: Multispectral classification of landsat-images using neural networks. *IEEE Transactions on Geoscience and Remote Sensing 30*, 3 (May 1992), 482–490.

[DM79] Delp E., Mitchell O.: Image compression using block truncation coding. *IEEE Transactions on Communications 27*, 9 (September 1979), 1335–1342.

[Don10] Donovan W.: Bc7 export, distributed with nvidia texture tools. http://code.google.com/p/nvidia-texture-tools/, 2010. Accessed on 1 July 2017.

[EN15] Edelsten A., NVIDIA: Astc compression gets a cuda boost. https://developer.nvidia.com/content/astc-compression-gets-cuda-boost, 2015.

[Fen03] Fenney S.: Texture compression using low-frequency signal modulation. In *HWWS '03: Proceedings of the ACM SIGGRAPH/EUROGRAPHICS Conference on Graphics Hardware* (2003), Eurographics Association, pp. 84–91.

[Fra99] Franzen R.: Kodak lossless true color image suite. http://r0k.us/graphics/kodak/, 1999. Accessed on 1 July 2017.

[INH99] Iourcha K. I., Nayak K. S., Hong Z.: System and method for fixed-rate block-based image compression with inferred pixel values. U. S. Patent 5956431, 1999.

[KB14] Kingma D. P., Ba J.: Adam: A method for stochastic optimization. CoRR abs/1412.6980, 2014.

[KLM13] Krajcevski P., Lake A., Manocha D.: Fastc: Accelerated fixed-rate texture encoding. In *I3D '13: Proceedings of the ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games* (New York, NY, USA, 2013), ACM, pp. 137–144.

[KM14a] Krajcevski P., Manocha D.: Real-time low-frequency signal modulated texture compression using intensity dilation. In *I3D '14: Proceedings of the 18th Meeting of the ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games* (New York, NY, USA, 2014), ACM, pp. 127–134.

[KM14b] Krajcevski P., Manocha D.: Segtc: Fast texture compression using image segmentation. In *HPG '14: Proceedings of High Performance Graphics* (Aire-la-Ville, Switzerland, Switzerland, 2014), Eurographics Association, pp. 71–77.

[KUMH17] Klambauer G., Unterthiner T., Mayr A., Hochreiter S.: Self-normalizing neural networks. arXiv preprint arXiv:1706.02515, 2017.

[NH10] Nair V., Hinton G. E.: Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML-10)* (2010), pp. 807–814.

[NLP*12] Nystad J., Lassen A., Pomianowski A., Ellis S., Olson T.: Adaptive scalable texture compression. In *HPG '12: Proceedings of the ACM SIGGRAPH/EUROGRAPHICS Conference on High Performance Graphics* (2012), Eurographics Association, pp. 105–114.

[OJ04] Oh K.-S., Jung K.: Gpu implementation of neural networks. *Pattern Recognition 37*, 6 (2004), 1311–1314.

[Ope10] OpenGL A. R. B.: ARB_texture_compression_bptc. http://www.opengl.org/registry/specs/ARB/texture_compression_bptc.txt, 2010. Accessed on 1 November 2016.

[Pix15] Pixar: Pixar one twenty eight. https://community.renderman.pixar.com/article/114/library-pixar-one-twenty-eight.html, 2015. Accessed on 1 July 2017.

[RB17] Rippel O., Bourdev L.: Real-time adaptive image compression. In *Proceedings of International Conference on Machine Learning* (2017).

[RDS*15] Russakovsky O., Deng J., Su H., Krause J., Satheesh S., Ma S., Huang Z., Karpathy A., Khosla A., Bernstein M., Berg A. C., Fei-Fei L.: ImageNet large scale visual recognition challenge. *International Journal of Computer Vision (IJCV) 115*, 3 (2015), 211–252.

[RGSS10] Rubinstein M., Gutierrez D., Sorkine O., Shamir A.: A comparative study of image retargeting. In *SIGGRAPH ASIA '10: ACM SIGGRAPH Asia 2010 Papers* (New York, NY, USA, 2010), ACM, pp. 160:1–160:10.

[SAM05] Ström J., Akenine-Möller T.: iPACKMAN: High-quality, low-complexity texture compression for mobile phones. In *HWWS '05: Proceedings of the ACM SIGGRAPH/EUROGRAPHICS conference on Graphics hardware* (2005), ACM, pp. 63–70.

[SCE01] Skodras A., Christopoulos C., Ebrahimi T.: The jpeg 2000 still image compression standard. *IEEE Signal Processing Magazine 18*, 5 (September 2001), 36–58.

[SHK*14] Srivastava N., Hinton G., Krizhevsky A., Sutskever I., Salakhutdinov R.: Dropout: A simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research 15*, 1 (2014), 1929–1958.

[SJ81] Shore J., Johnson R.: Properties of cross-entropy minimization. *IEEE Transactions on Information Theory 27*, 4 (July 1981), 472–482.

[SP07] Ström J., Pettersson M.: ETC2: Texture compression using invalid combinations. In *GH '07: Proceedings of the 22nd ACM SIGGRAPH/EUROGRAPHICS Symposium on Graphics Hardware* (2007), Eurographics Association, pp. 49–54.

[SRASC14] Sharif Razavian A., Azizpour H., Sullivan J., Carlsson S.: Cnn features off-the-shelf: An astounding baseline for recognition. In *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops* (June 2014).

© 2018 The Authors
Computer Graphics Forum © 2018 The Eurographics Association and John Wiley & Sons Ltd.

14678659, 2019, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cgf.13534 by Nanjing University Of Science, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

[Stu17] Studios B. G.: Fallout 4 - high resolution texture pack. http://store.steampowered.com/app/540810/Fallout_4__High_Resolution_Texture_Pack/, 2017. Accessed on 1 October 2017.

[TJZ*17] Tao W., Jiang F., Zhang S., Ren J., Shi W., Zuo W., Guo X., Zhao D.: An end-to-end compression framework based on convolutional neural networks. In *Proceedings of 2017 Data Compression Conference (DCC)* (April 2017), pp. 463–463.

[TVJ*16] Toderici G., Vincent D., Johnston N., Hwang S. J., Minnen D., Shor J., Covell M.: Full resolution image compression with recurrent neural networks. CoRR abs/1608.05148, 2016.

[vEB81] van Emde Boas P.: *Another Np-complete Partition Problem and the Complexity of Computing Short Vectors in a Lattice*. Mathematical Preprints Series. Mathematisch Instituut, Universiteit van Amsterdam, Amsterdam, 1981.

[Wal92] Wallace G. K.: The jpeg still picture compression standard. *IEEE Transactions on Consumer Electronics 38*, 1 (February 1992), xviii–xxxiv.

[Wav06] Waveren J. M. P. V.: Real-time DXT compression. *Intel Software Network* (2006).

[WBX*07] Wu S. G., Bao F. S., Xu E. Y., Wang Y. X., Chang Y. F., Xiang Q. L.: A leaf recognition algorithm for plant classification using probabilistic neural network. In *Proceedings of 2007 IEEE International Symposium on Signal Processing and Information Technology* (December 2007), pp. 11–16.

[WCn07] Waveren J. M. P. V., Castaño I.: Real-time YCoCg-DXT compression. *NVIDIA Developer Network* (2007).

**Supporting Information**

Additional supporting information may be found online in the Supporting Information section at the end of the article.

**Table S1**. The number of layers and number of neurons in each layer for P-NN network (Section 3.4).

**Table S2**. The number of layers and number of neurons in each layer for DEM-NN network (Section 3.4).

© 2018 The Authors
Computer Graphics Forum © 2018 The Eurographics Association and John Wiley & Sons Ltd.

14678659, 2019, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cgf.13534 by Nanjing University Of Science, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License